

The PR you would have open yourself

+ 61 TL;DR The advent of code agents What does this have to do with MLX?

来源：<https://huggingface.co/blog/transformers-to-mlx>

分类：07_计算机视觉与NLP概览

+ 61

TL;DR The advent of code agents What does this have to do with MLX?
it Test harness How to use the Skill Next steps and known shortcomings What's next What doesn't work yet Conclusion Resources Thank you!
mlx-lm using a Skill and test harness

TL;DR

The advent of code agents

What does this have to do with MLX?

What we did

How we did it

Test harness

How to use the Skill

Next steps and known shortcomings What's next What doesn't work yet

What's next

What doesn't work yet

Conclusion

Resources

Thank you!

TL ; DR

We provide a Skill and a test harness to help port mlx-lm, so they become (almost) instantly available transformers. The Skill is designed to support code automation. We explain why we did it, how, and code to open source in the age of agents.

The advent of code agents

In 2026, code agents started to actually work. What if your editor turned into a system that one-shot specifications. The generated code usually works and makes reasonable assumptions about details you Huang puts it, we've instantly gone from 30 million Creative minds are unleashed.

But it forces us to rethink open source.

Take the transformers library as an example. It has thousands of projects, has been downloaded over a agent can instruct it to find some open issue, find what's happening. Those people feel happy because but the sad reality is that, most of the time, the

Why not? There are two assumptions that agent-generated

Codebases like transformers care deeply about the doesn't matter what the code looks like, but transformers thousands of people, transformers is primarily built method, through code. Model files read top to bottom understand them without jumping through complex API library design and is the reason why, for example

Codebases like transformers care deeply about the doesn't matter what the code looks like, but transformers thousands of people, transformers is primarily built method, through code. Model files read top to bottom understand them without jumping through complex API library design and is the reason why, for example

Agents don't have that context. Because design decisions refactors to "improve" the codebase by following are breaking implicit contracts between the library generalize too early, don't notice when a change break performance. They are also sycophantic, and through diligently, including ones a maintainer v

comment .

Agents don't have that context. Because design decisions are made by refactoring to "improve" the codebase by following the design, they are breaking implicit contracts between the library and its users. They generalize too early, don't notice when a change breaks performance. They are also sycophantic, and they don't think through things as diligently, including ones a maintainer would think of. comment .

A small number of maintainers still has to read every PR to make sure the design direction is right, identify side effects, and make sure the code is tenfold, but the amount of maintainers has not (and probably never will) scale).

What does this have to do with MLX?

Transformers is one of the first projects to feel the same dynamic is happening everywhere. As an expert reviewer, you are swamped because anyone can now build a PR.

The same logic applies to MLX: their maintainers read every PR carefully. We wanted to see whether agents could help model ports fast, and at the same time support reviewers who aspire to produce PRs that could have come from a maintainer. We provide additional artifacts to increase the signal-to-noise ratio in comparisons, and a separate non-agentic test harness.

Another connection between transformers and MLX is that MLX is ported from transformers implementations. Because of its high readability, it has become the source of truth for many contributors wait until the transformers implementation is ready for frameworks. As a side effect, this is an excellent example of how MLX naturally limits the scope: rather than creating a new framework, it relies on transformers code as the source of truth.

This approach supports our goal: when a model lands on MLX shortly after.

What we did

We built a Skill that mlx-lm contributors can use. Given a prompt like "convert the olmo_hybrid architecture to a virtual environment to work on, discovers and downloads the transformers modeling code, writes the tests. If results don't look right, it debugs and iterates until it's satisfied.

We designed it to be useful to reviewers as much

For the contributor, the Skill of course handles on the Hub, diffing their configs to spot parameter downloading checkpoints, setting up editable installs, it also handles the more difficult modeling tasks, details and verifies sensitive areas, like RoPE or hard-to-find bugs. It detects when the config does safetensors metadata header. It runs per-layer comparison pinpoint exactly where divergence occurs. These are with porting experience would think to run.

For the reviewer, the Skill produces a PR that is does look like a careful human submission. Review conventions: idiomatic solutions, no unnecessary modifications to shared utilities without explicit agent-assisted, we try to include more data than possible. The PR body includes a report with a summary architectural differences, generation examples, per-layer comparisons against the transformers base agent-assisted, and the Skill will not open it until results.

For verification, the Skill generates a test manifest harness that is, by design, easily reproducible and complacency (more on this below).

How we did it

Skills are recipes for agents: simple text files through a complex task. They are not magic; you need and iteration. But they provide consistency (even different people would prompt differently), minimizing anyone can read the Skill to understand what it does improvements.

We bootstrapped the Skill by porting a model ourselves it to port GLM 4.7 from transformers to mlx-lm, generating normal session. One trick: I pointed Claude at a the already-existing implementation, so I could compare. After a few iterations I had a working implementation. Claude approached the problem, and the first draft summary of the process. I edited it heavily, and @gabegoodhart, who kindly shared their own porting

We repeated this loop several times and the Skill stuff such as RoPE bugs that may produce plausible

float32 precision contamination that silently kills frequently these things happen!), config fields to implementation must handle, distributed inference on a single machine. We taught it how to invoke the hf client. Importantly, we instructed it to run the tests that declare success until they pass.

Source: @Prince_Canuma

On the cultural side, we covered softer character traits to make a PR easy to review: don't use comments to explain code and the code (♂), never propose refactoring asking. These rules cost the agent nothing but save

The end result: the contributor types a prompt, a test harness plus a test manifest for the external test harness

Test harness

The Skill shares a comprehensive results report as the agent runs during conversion, but we didn't want to accept them. To go a step forward, we created a systematic tests on the converted code. This brings

Removes uncertainty about the LLM hallucinating results

Guarantees reproducibility: anyone can download the

Documentation and transparency. All results are saved per-model details, raw inputs/outputs saved as JSON results folders so we know what we ran even if we

The test harness is not a CI gate. Some checks are correct?), but most are qualitative. Is it normal for long sequences? Is a 4% relative logits difference acceptable? These are judgement calls based on experience. The harness provides useful signal, but it's the reviewer's call.

How to use the Skill

The Skill is designed for the people who are already doing it manually on their own. It's not meant for novices rarely accepted on sight. The typical cycle is: code improvements, both sides iterate until the quality of submissions, it will remain true for agent-assisted

If you're not prepared to engage in that cycle, your reviewers will make an effort to understand your code, so you should do the same. Own the code, and be more particular, don't hand reviewer comments back to LLMs double down on their decisions, go on tangent. When you engage with the reviewer, this becomes a personal turn to discuss and be respectful of the time the reviewer

You can also use the Skill to learn; you don't need a lot of and experience build. Read the Skill to identify what it contains nearly 15 thousand words among the skill. Point it to your own fork of mlx-lm, try a conversation, and an accepted implementation once it lands in the official repo, you'll learn a lot about transformers, MLX, and LLMs.

If you're ready:

We developed and tested the Skill using Claude Code or other coding agents, but we haven't tested the Skill in a real environment, please let us know how it goes!

Next steps and known shortcomings

The Skill works well for LLMs in mlx-lm, but there are some known shortcomings.

What's next

mlx-vlm. Vision-language models live in a separate repo, so the modeling code, mlx-vlm requires processors to run, so it sees the input. We're looking forward to collaborating with what he does.

llama.cpp. Some of the same challenges apply. Porting to be replicated in C++, and numerical difference between Python and C++ tightly scoped agent might help.

The test harness. We want to expand the test battery to run tests automatically on our infra.

What doesn't work yet

Shared utilities in mlx-lm. mlx-lm is less strict about code patterns into shared functions. The Skill is purposeful about model files (same as transformers), but reviewers want to move repeated code into shared modules.

VLMs and other architectures, as noted above.

Quantized model uploads. The Skill tests quantization on the Hub. We think it doesn't make sense to upload quantized models, but we could create a flow to do it later.

Thinking tests. No thinking-specific tests have been added yet, but we will add and verify generations from these models, but won't be able to verify them.

Conclusion

The bottleneck in open source is not typing speed, but the ability to type it without breaking the implicit and explicit context. We are exploring this process, if we teach them what matters. We explore this with mlx-lm, and hope it's useful for contributors and users to make conversions faster!

Resources

Contributions:

[transformers-to-mlx](#) Skill repo

[Test Harness](#) repo

[Example agent-assisted conversion against a fork](#)

The libraries:

[mlx-lm](#), the target library

[transformers](#), the source of truth for modeling code

Background:

[Claude Code Skills docs](#)

[Transformers design philosophy](#)

[The Transformers Library: standardizing model definitions](#)

Thank you!

Thanks a lot to Ben, Shaun, Aritra for reading post
so much better

We are incredibly indebted to Apple for making ML
community for instantly recognizing its value and

More Articles from our Blog

DeepSeek-V4: a million-token context that agents

AI and the Future of Cybersecurity: Why Openness

Community

· Sign up or log in to comment

+ 55